



Security Review Report for SquadSwap

August 2026



Table of Contents

1. About Hexens

2. Executive summary

3. Security Review Details

- Security Review Lead
- Scope
- Changelog

4. Severity Structure

- Severity characteristics
- Issue symbolic codes

5. Findings Summary

6. Weaknesses

- CL exactOut swaps can take the maximum input for no output
- Missing pool-manager validation enables LP fee manipulation
- App reserve check can revert legitimate JIT swaps
- beforeSwap sends the owner's fee share before the swap
- Missing CL action check allows theft of fees from Router-approved positions
- Repeated V2 pairs can deliver less than the requested exact output
- Missing share slippage protection allows LP value extraction
- ManagerFeeHookAdmin cannot manage factory-created hooks
- Unresolved Bin recipients can permanently freeze liquidity shares
- CL migration rebates can be stolen
- ManagerFeeHook.afterBurn returns the wrong selector

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for SquadSwap protocol, a fork of PancakeSwap Infinity that adds more functionality, such as the Automated Liquidity Manager (ALM).

This report includes the findings on the SquadSwap core code.

Our security assessment was a full review of the code, spanning a total of 1.5 weeks.

During our review, we identified multiple high severity vulnerability, which could have resulted in loss of assets.

We also identified some minor severity vulnerabilities and code optimizations.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- **Review Led by**

Kasper Zwijsen, Head of Audits

- **Scope**

The analyzed resources are located on:

<https://github.com/skyrocktech/SquadSwapV4/tree/6b5625690edcf3e411d846e75fbf1dd62b1e9391>

The issues described in this report were fixed in the following commit:

<https://github.com/skyrocktech/SquadSwapV4/tree/c854b9949f3fe002a9f418a3d30d2e82298f4410>

- **Changelog**

■ 17 August 2026	Audit start
■ 1 September 2026	Initial report
■ 18 September 2026	Revision received
■ 22 September 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

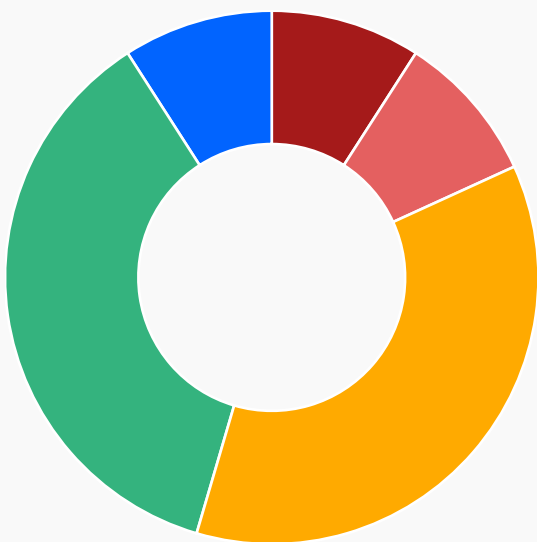
Severity

Number of findings

Critical	1
High	1
Medium	4
Low	4
Informational	1

Total:

11



- Critical
- High
- Medium
- Low
- Informational



Fixed

6. Weaknesses

This section contains the list of discovered weaknesses.

SQUAD1-5 | CL exactOut swaps can take the maximum input for no output

Fixed 

Severity:

Critical

Probability:

Likely

Impact:

Critical

Path:

```
squadswap-aug-26/squadswap-v4-periphery/src/pool-cl/CLRouterBase.sol:  
_swapExactOutputSingle#L93-L118  
squadswap-aug-26/squadswap-v4-periphery/src/pool-cl/CLRouterBase.sol:  
_swapExactOutput#L120-L163
```

Description:

These functions execute single-hop and multi-hop exact-output swaps through the CL pool manager. A user specifies the exact amount they want to receive and caps the input with **amountInMaximum**. The router calls the pool, then settles the resulting input debt and transfers the output credit to the user.

For an exact-output swap, the call should either deliver the full requested output within the input cap or revert. This is what lets a user treat **amountInMaximum** as slippage protection rather than an amount they may pay for any output.

However, a CL pool can stop at its price limit while part of the requested output remains unfilled. The router sets this limit near the global minimum or maximum price, but after the swap it only checks the input:

```
if (amountIn > params.amountInMaximum) {  
    revert ISquadSwapV4Router.TooMuchRequested(...);  
}
```

It never checks that the returned output equals **params.amountOut**. The normal open-delta settlement then charges the actual input and sends only the actual output. In a multi-hop swap, a partial fill is also passed into the previous hop as if it were the required amount, so the same issue applies.

An adversarial LP can exploit this as follows:

1. The LP supplies normal liquidity so a victim receives a valid quote for an exact-output swap.
2. Before the victim's swap, the LP removes that liquidity and adds a small, one-sided position extending toward an extreme price.
3. The victim's swap crosses this position, paying close to **amountInMaximum** while receiving only a tiny part of the requested output, then stops at the router's price limit.
4. The router accepts the partial fill because the input stayed within the cap. The LP removes the crossed position and recovers the victim's input.

As a result, the victim can lose almost their entire maximum input while receiving only dust.

```
function _swapExactOutputSingle(
  CLSwapExactOutputSingleParams calldata params
) internal {
  uint128 amountOut = params.amountOut;
  if (amountOut == ActionConstants.OPEN_DELTA) {
    amountOut = _getFullDebt(
      params.zeroForOne
        ? params.poolKey.currency1
        : params.poolKey.currency0
    ).toUint128();
  }
  uint128 amountIn = (
    _swapExactPrivate(
      params.poolKey,
      params.zeroForOne,
      int256(uint256(amountOut)),
      params.hookData
    )
  ).toUint128();
  if (amountIn > params.amountInMaximum) {
    revert ISquadSwapV4Router.TooMuchRequested(
      params.amountInMaximum,
      amountIn
    );
  }
}
```

Remediation:

After every exact-output pool call, verify that the actual output equals the requested output and revert on a short fill. Apply this check to every hop before carrying its input amount into the previous hop.

SQUAD1-1 | Missing pool-manager validation enables LP fee manipulation

Fixed 

Severity:

High

Probability:

Likely

Impact:

High

Path:

CLFeeManagerHook.sol#L62, BinFeeManagerHook.sol#L62, ManagerFeeHook.sol#L187 and the other hook callbacks.

Description:

The hook callbacks do not verify that the caller is the pool manager. As a result, anyone can directly call `afterInitialize()`

```
function afterInitialize(...) external override returns (bytes4) {
    PoolId poolId = key.told();

    // Set default fee to 0.05%
    uint24 defaultFee = 500;
    poolFees[poolId] = defaultFee;

    // Update fee in pool manager
    poolManager.updateDynamicLPFee(key, defaultFee);

    emit FeeUpdated(poolId, defaultFee);

    return IBinHooks.afterInitialize.selector;
}
```

The callback then resets the pool's dynamic LP fee to the hardcoded 500 (0.05%) and calls `poolManager.updateDynamicLPFee()`.

This bypasses the owner-only fee setters, allowing an attacker to repeatedly reset a pool's fee and execute a swap at the lower fee. For example, a 10% fee can be reduced to 0.05%, allowing the attacker to keep the difference that would otherwise go to LPs.

Remediation:

Add an `onlyPoolManager` check to every `before*` and `after*` callback in all three hooks, including `ManagerFeeHook.beforeSwap()`.

SQUAD1-3 | App reserve check can revert legitimate JIT swaps

Fixed 

Severity:

Medium

Probability:

Unlikely

Impact:

Medium

Description:

During a swap, **afterSwap** executes before the swap's input is added to the app's reserve. If a JIT hook removes its liquidity during **afterSwap**, the withdrawal is deducted from the reserve immediately, while the swap input is still not accounted for.

This can cause the reserve to temporarily appear insufficient and make the transaction revert, even though the swap input would fully cover the withdrawal once the swap finishes.

Impact: JIT liquidity hooks can fail on pools without enough resident liquidity to cover this temporary gap.

Remediation:

Allow the app's reserve to temporarily go negative during a lock and track the shortfall as a transient deficit. Any deficit must be fully repaid before the lock ends; otherwise the transaction reverts.

```
if (reserve >= amount) {
    reservesOfApp[msg.sender][currency] = reserve - amount;
} else {
    reservesOfApp[msg.sender][currency] = 0;
    AppDeficit.add(
        msg.sender,
        currency,
        amount - reserve
    );
}
```

Deposits repay the deficit before increasing the stored reserve:

```
uint256 remaining =
    AppDeficit.repay(msg.sender, currency, uint128(-delta));

if (remaining != 0) {
    reservesOfApp[msg.sender][currency] += remaining;
}
```

Finally, **lock()** checks that no deficit remains:

```
if (AppDeficit.count() != 0)
    revert AppCurrencyNotFullyRepaid();
```

This preserves cross-app reserve protection while allowing legitimate JIT re-entry to complete.

SQUAD1-6 | beforeSwap sends the owner's fee share before the swap

Fixed 

Severity:

Medium

Probability:

Unlikely

Impact:

Medium

Path:

ManagerFeeHook.sol#L139-L148

Description:

beforeSwap() pays the owner's fee share using **vault.take()** before the swapper has settled the swap:

```
vault.burn(address(this), specifiedCurrency, ownerFeeAmount);  
vault.take(specifiedCurrency, owner(), ownerFeeAmount);
```

For an exact-input swap, the fee is taken from the input token, which is not in the vault yet. Therefore, the vault must already have enough of that token to front the fee.

If the pool is one-sided and the vault has not enough balance of the input token, **vault.take()** reverts and the swap cannot execute.

SQUAD1-10 | Missing CL action check allows theft of fees from Router-approved positions

Fixed ✓

Severity:

Medium

Probability:

Unlikely

Impact:

Medium

Path:

```
squadswap-v4-universal-router/src/modules/V3ToSquadSwapV4Migrator.sol:  
_checkSquadCIPositionManagerCall#L92-L126
```

Description:

The Universal Router uses `_checkSquadCIPositionManagerCall` to validate calls forwarded to the CL Position Manager. These calls come through `SQUAD_CL_POSITION_CALL`, and the check is meant to allow migration-related minting while preventing callers from changing existing positions that have approved the Router.

Calls that act on an existing position should be rejected unless the external caller is also authorized for that position. Otherwise, the Position Manager sees the Universal Router as the caller, so a standing token approval for the Router would be enough to pass its ownership check. However, the validator only blocks `CL_INCREASE_LIQUIDITY`, `CL_DECREASE_LIQUIDITY`, and `CL_BURN_POSITION`. It misses `CL_INCREASE_LIQUIDITY_FROM_DELTAS`, which also acts on an existing token ID:

```
if (  
    action == Actions.CL_INCREASE_LIQUIDITY ||  
    action == Actions.CL_DECREASE_LIQUIDITY ||  
    action == Actions.CL_BURN_POSITION  
) {  
    revert OnlyMintAllowed();  
}
```

With no currency credit available, `CL_INCREASE_LIQUIDITY_FROM_DELTAS` calculates a zero liquidity increase. For a nonempty position, this still realizes its accrued fees. The attacker can then include the allowed `TAKE_PAIR` action in the same call and send those fees to themselves. This affects Router-approved positions in pools without hooks, as well as pools whose hooks allow the zero-liquidity update without redirecting the fee credit.

An attack works as follows:

1. A victim has a CL position with accrued fees and has approved the Universal Router for the token, or for all their CL position NFTs.
2. An attacker calls the Router with **SQUAD_CL_POSITION_CALL**, passing **CL_INCREASE_LIQUIDITY_FROM_DELTAS(tokenId, 0, 0, hookData)** followed by **TAKE_PAIR(currency0, currency1, attacker)**.
3. The missing action check lets the call through, and the victim's Router approval satisfies the Position Manager's approval check.
4. The zero-liquidity update realizes the position's fees, and **TAKE_PAIR** transfers them to the attacker. The attack can be repeated as more fees accrue.

```
function _checkSquadCIPositionManagerCall(
    bytes calldata inputs
) internal view {
    bytes4 selector;
    assembly {
        selector := calldataload(inputs.offset)
    }
    if (selector != SQUAD_CL_POSITION_MANAGER.modifyLiquidities.selector) {
        revert InvalidAction(selector);
    }

    // slice is `abi.encode(bytes unlockData, uint256 deadline)`
    bytes calldata slice = inputs[4:];
    // the first bytes(0) extracts the unlockData parameter from modifyLiquidities
    // unlockData = `abi.encode(bytes actions, bytes[] params)`
    // the second bytes(0) extracts the actions parameter from unlockData
    bytes calldata actions = slice.toBytes(0).toBytes(0);

    uint256 numActions = actions.length;

    for (uint256 actionIndex = 0; actionIndex < numActions; actionIndex++) {
        uint256 action = uint8(actions[actionIndex]);

        if (
            action == Actions.CL_INCREASE_LIQUIDITY ||
            action == Actions.CL_DECREASE_LIQUIDITY ||
            action == Actions.CL_BURN_POSITION
        ) {
            revert OnlyMintAllowed();
        }
    }
}
```

Remediation:

Reject **CL_INCREASE_LIQUIDITY_FROM_DELTAS** as an existing-position action. Preferably, replace the denylist with a strict allowlist of the minting and settlement actions required by the migration flow.

SQUAD1-11 | Repeated V2 pairs can deliver less than the requested exact output

Fixed ✓

Severity:

Medium

Probability:

Unlikely

Impact:

Medium

Path:

squadswap-aug-26/squadswap-v4-universal-router/src/modules/squadswap/v2/V2SwapRouter.sol:
v2SwapExactOutput#L113-L131

Description:

The **v2SwapExactOutput** function lets users specify an exact amount of output tokens and the maximum input they are willing to spend. It first calls **getAmountInMultihop** to quote the required input, transfers that input to the first pair, and then executes each swap through **_v2Swap**.

For a normal route, the quoted reserves match the reserves used during execution. The recipient should therefore receive at least **amountOut**, or the transaction should revert without spending the user's tokens.

This breaks when the route visits the same V2 pair more than once. **getAmountInMultihop** quotes every hop before execution, so both visits use the pair's initial reserves. The first swap then changes those reserves, and **_v2Swap** calculates the later swap using the updated values. **v2SwapExactOutput** only checks the stale input quote against **amountInMaximum**; it never checks how many tokens the recipient actually received.

For example, consider three pairs with 1,000 units on each side and the route **A -> B -> C -> A -> B**. Requesting 100 B produces a quote of 172 A. The first A/B swap changes that pair's reserves, so the final A/B swap returns only about 74 B. The call still succeeds and spends all 172 A.

An exploit scenario is:

1. A user receives or constructs a route such as **A -> B -> C -> A -> B**, which uses the A/B pair twice.
2. The user calls the V2 exact-output command for 100 B with an input limit of at least 172 A.
3. The router quotes both A/B hops from the same initial reserves and transfers 172 A from the user.
4. The first A/B hop updates the pair's reserves, causing the final hop to return only about 74 B. Since there is no final-output check, the transaction succeeds.

```
function v2SwapExactOutput(  
    address recipient,  
    uint256 amountOut,  
    uint256 amountInMaximum,
```

```

    address[] calldata path,
    address payer
) internal {
    (uint256 amountIn, address firstPair) = UniversalRouterHelper
        .getAmountInMultihop(
            SQUADSWAP_V2_FACTORY,
            SQUADSWAP_V2_PAIR_INIT_CODE_HASH,
            amountOut,
            path
        );
    if (amountIn > amountInMaximum) revert V2TooMuchRequested();

    payOrPermit2Transfer(path[0], payer, firstPair, amountIn);
    _v2Swap(path, recipient, firstPair);
}

```

Remediation:

Measure the recipient's final-token balance before and after **_v2Swap**, and revert if the increase is below **amountOut**. Repeated pairs should also be rejected, or their reserve changes must be simulated when calculating the input quote.

SQUAD1-4 | Missing share slippage protection allows LP value extraction

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Medium

Description:

When adding liquidity, **BinPositionManager** only protects the input side of the transaction:

```
delta.validateMaxIn(amount0Max, amount1Max);
```

and verifies that the active bin remains within the user's **idSlippage** tolerance.

However, it never validates the output side, the amount of LP shares actually minted. As a result, users can pay the expected amount of tokens while receiving significantly fewer shares than anticipated.

In a Bin pool, shares minted depend on the current reserve composition of the active bin. An attacker can sandwich a liquidity-add transaction by first swapping assets to skew the bin's composition. The victim's deposit then becomes poorly matched to the bin reserves, causing a larger implicit swap and composition fee during minting. Although the victim spends the same amount of tokens and the active bin ID remains within tolerance, fewer LP shares are minted.

The root cause is that the contract never validates the result returned by **binPoolManager.mint()**:

```
(BalanceDelta delta, BinPool.MintArrays memory mintArray) = binPoolManager.mint(...);
```

While **mintArray.liquidityMinted** contains the actual shares minted for each bin, the value is only used for minting and bookkeeping and is never checked against a user-specified minimum.

Without a minimum-share guarantee, users are exposed to silent value loss from MEV sandwich attacks even when all existing slippage checks pass.

Remediation:

Introduce a user-supplied minimum liquidity parameter and validate the minted shares immediately after **mint()** returns:

```
if (mintArray.liquidityMinted[i] < minLiquidities[i]) {  
    revert LiquiditySlippageCaught(...);  
}
```

This provides output-side slippage protection similar to **amountOutMin** checks in swap functions and prevents liquidity additions from succeeding when the number of shares received falls below the user's expected minimum.

SQUAD1-7 | ManagerFeeHookAdmin cannot manage factory-created hooks

Fixed ✓

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

ManagerFeeHookAdmin.sol, ManagerFeeHook.sol, ManagerFeeHookFactory.sol

Description:

ManagerFeeHookAdmin is intended to manage hook configuration, but its **setSwapFee()** and **setFeeRecipient()** functions cannot modify hooks created through **ManagerFeeHookFactory**.

When a hook is created, the factory sets the admin and then transfers ownership to the user:

```
hook.setAdmin(defaultAdmin);  
hook.transferOwnership(msg.sender);
```

The problem is that the hook's **setSwapFee()** and **setFeeRecipient()** functions use **onlyOwner**:

```
// ManagerFeeHook  
function setSwapFee(...) external onlyOwner { ... }  
function setFeeRecipient(...) external onlyOwner { ... }  
  
// ManagerFeeHookAdmin  
IManagerFeeHook(hook).setSwapFee(key, newFee); // calls as not owner
```

When **ManagerFeeHookAdmin** calls these functions, **msg.sender** is the **ManagerFeeHookAdmin** contract, not the hook owner. Therefore, both calls revert with **OwnableUnauthorizedAccount**.

Remediation:

If hook owners should control their own fee recipients, keep **setFeeRecipient()** owner-only. Add a separate **onlyOwnerOrAdmin** authorization to **setSwapFee()** so the protocol can enforce a fee limit or correct an unsafe fee configuration.

SQUAD1-8 | Unresolved Bin recipients can permanently freeze liquidity shares

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

```
squadswap-aug-26/squadswap-v4-periphery/src/pool-bin/BinPositionManager.sol:  
_handleAction#L151-L195  
squadswap-aug-26/squadswap-v4-periphery/src/pool-bin/BinPositionManager.sol:  
_addLiquidity#L287-L383
```

Description:

The Bin Position Manager handles **BIN_ADD_LIQUIDITY** and **BIN_ADD_LIQUIDITY_FROM_DELTAS** by adding a user's tokens to a pool and minting the resulting liquidity shares to the requested recipient. These actions are available to regular users and integrations through the shared action-router API.

That API uses **ActionConstants.MSG_SENDER** (**address(1)**) and **ActionConstants.ADDRESS_THIS** (**address(2)**) as shortcuts for the action executor and the Position Manager. The recipient should be resolved with **_mapRecipient** before any shares are minted, as it is in the CL Position Manager.

However, both Bin action branches pass **liquidityParams.to** directly to **_addLiquidity**. The function then mints shares to that raw address:

```
_mint(to, tokenId, mintArray.liquidityMinted[i]);
```

If a caller uses **MSG_SENDER**, the deposit succeeds but the shares are recorded under **address(1)** instead of the caller. This is also the value used by the repository's own **AddLiquidityBin.s.sol** script. The user still pays the pool's token debt, while nobody can transfer or burn the shares because doing so requires permission from their recorded holder. Since **address(1)** cannot grant that permission, the deposited principal and future fees are permanently inaccessible. **ADDRESS_THIS** is affected in the same way and is left as the literal **address(2)**.

An affected flow works as follows:

1. A user submits either Bin add-liquidity action with **to** set to **ActionConstants.MSG_SENDER**.
2. The Position Manager adds the user's tokens to the pool and settles the plan normally.
3. **_addLiquidity** mints the position shares to the literal **address(1)**.

4. The user cannot transfer or remove the shares, permanently freezing their deposit and any fees it earns.

```
function _addLiquidity(
    PoolKey calldata poolKey,
    uint128 amount0,
    uint128 amount1,
    uint128 amount0Max,
    uint128 amount1Max,
    uint256 activeIdDesired,
    uint256 idSlippage,
    int256[] calldata deltaIds,
    uint256[] calldata distributionX,
    uint256[] calldata distributionY,
    address to,
    bytes calldata hookData
) internal {
    uint256 deltaLen = deltaIds.length;
    uint256 lenX = distributionX.length;
    uint256 lenY = distributionY.length;
    assembly ("memory-safe") {
        /// @dev revert if deltaLen != lenX || deltaLen != lenY
        if iszero(and(eq(deltaLen, lenX), eq(deltaLen, lenY))) {
            mstore(0, 0xaaad13f7) // selector InputLengthMismatch
            revert(0x1c, 0x04)
        }
    }

    if (
        activeIdDesired > type(uint24).max || idSlippage > type(uint24).max
    ) {
        revert AddLiquidityInputActiveIdMismatch();
    }

    /// @dev Checks if the activeId is within slippage before calling mint. If user mint to activeId and there
    // was a swap in hook.beforeMint() which changes the activeId, user txn will fail
    (uint24 activeId, , ) = binPoolManager.getSlot0(poolKey.told());
    if (activeIdDesired + idSlippage < activeId) {
        revert IdSlippageCaught(activeIdDesired, idSlippage, activeId);
    }
    if (activeIdDesired - idSlippage > activeId) {
        revert IdSlippageCaught(activeIdDesired, idSlippage, activeId);
    }

    bytes32[] memory liquidityConfigs = new bytes32[](deltaLen);
    for (uint256 i; i < liquidityConfigs.length; i++) {
```

```

int256 _id = int256(uint256(activeId)) + deltaIds[i];
if (_id < 0 || uint256(_id) > type(uint24).max)
    revert IdOverflows(_id);

liquidityConfigs[i] = LiquidityConfigurations.encodeParams(
    uint64(distributionX[i]),
    uint64(distributionY[i]),
    uint24(uint256(_id))
);
}

bytes32 amountIn = amount0.encode(amount1);
(
    BalanceDelta delta,
    BinPool.MintArrays memory mintArray
) = binPoolManager.mint(
    poolKey,
    IBinPoolManager.MintParams({
        liquidityConfigs: liquidityConfigs,
        amountIn: amountIn,
        salt: bytes32(0)
    }),
    hookData
);

/// Slippage checks, similar to CL type. However, this is different from TJ. In SS squadswap,
/// as hooks can impact delta (take extra token), user need to be protected with amountMax instead
delta.validateMaxIn(amount0Max, amount1Max);

// mint
PoolId poolId = cachePoolKey(poolKey);
uint256[] memory tokenIds = new uint256[](mintArray.ids.length);
for (uint256 i; i < mintArray.ids.length; i++) {
    uint256 tokenId = poolId.toTokenId(mintArray.ids[i]);
    _mint(to, tokenId, mintArray.liquidityMinted[i]);

    if (_positions[tokenId].binId == 0) {
        _positions[tokenId] = TokenPosition({
            poolId: poolId,
            binId: uint24(mintArray.ids[i])
        });
    }

    tokenIds[i] = tokenId;
}

```

```
emit TransferBatch(  
    msgSender(),  
    address(0),  
    to,  
    tokenIds,  
    mintArray.liquidityMinted  
);  
}
```

Remediation:

Call `_mapRecipient(liquidityParams.to)` in both Bin add-liquidity branches before passing the recipient to `_addLiquidity`. Add regression tests for both supported sentinels and consider rejecting unresolved reserved addresses during minting.

SQUAD1-9 | CL migration rebates can be stolen

Fixed ✓

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

squadswap-aug-26/squadswap-v4-periphery/src/pool-cl/CLMigrator.sol:_addLiquidityToTargetPool
#L166-L253

squadswap-aug-26/squadswap-v4-periphery/src/base/BaseMigrator.sol:withdrawLiquidityFromV3
#L153-L203

squadswap-aug-26/squadswap-v4-periphery/src/pool-cl/CLPositionManager.sol:_sweep
#L588-L592

Description:

The CL migration flows withdraw a user's old liquidity, mint a new CL position, and refund any tokens that were not used. `_addLiquidityToTargetPool` calculates the expected token usage from the pool price and position range before calling the CL Position Manager.

The refund should be based on the amount the Position Manager actually spends. This matters because a supported `afterAddLiquidity` return-delta hook can give the user a rebate and reduce the mint debt. If the regular mint costs D tokens and the hook rebates R , only $D - R$ should be treated as consumed.

However, the migrator returns the pre-hook estimate D as the consumed amount and calculates the refund as:

$\text{amountIn} - D$

The correct refund is $\text{amountIn} - (D - R)$. For an ERC20, the missing R stays in the migrator. For native currency, the migrator sends D to the Position Manager, which settles only $D - R$, leaving R in the Position Manager.

Both leftovers can be stolen. Anyone can submit a **SWEEP** action to the CL Position Manager and take its full native balance. For ERC20s, the migration functions accept a caller-supplied legacy pair or V3 position manager and trust its reported withdrawal amounts without checking balance changes. A fake source can therefore report the stranded balance as newly received funds, then make the migrator send that balance back as a refund.

An attack works as follows:

1. A victim migrates into a CL pool whose valid add-liquidity hook rebates R of one currency.

2. The Position Manager spends only **D - R**, but the migrator treats **D** as spent, so the victim does not receive **R**.
3. If the currency is native, an attacker calls the public Position Manager with a **SWEEP** action and receives the leftover ETH.
4. If it is an ERC20, the attacker uses a fake legacy source that reports **R** without transferring it, then mints a small one-sided position with the other token. The migrator treats the stranded tokens as unused source funds and refunds them to the attacker.

```
function _addLiquidityToTargetPool(
    MintParams memory params,
    uint256 deadline
) internal returns (uint256 amount0Consumed, uint256 amount1Consumed) {
    /// @dev currency1 cant be NATIVE
    bool nativePair = params.poolKey.currency0.isNative();
    if (!nativePair) {
        permit2ApproveMaxIfNeeded(
            params.poolKey.currency0,
            address(cIPositionManager),
            params.amount0In
        );
    }
    permit2ApproveMaxIfNeeded(
        params.poolKey.currency1,
        address(cIPositionManager),
        params.amount1In
    );

    (uint160 sqrtPriceX96, int24 activeTick, , ) = cIPoolManager.getSlot0(
        params.poolKey.tolid()
    );
    uint160 sqrtRatioAX96 = TickMath.getSqrtRatioAtTick(params.tickLower);
    uint160 sqrtRatioBX96 = TickMath.getSqrtRatioAtTick(params.tickUpper);
    uint128 liquidity = LiquidityAmounts.getLiquidityForAmounts(
        sqrtPriceX96,
        sqrtRatioAX96,
        sqrtRatioBX96,
        params.amount0In,
        params.amount1In
    );

    if (liquidity < params.liquidityMin) {
        revert INSUFFICIENT_LIQUIDITY();
    }
}
```

```

// Calculate amt0/amt1 from liquidity, similar to CLPool modifyLiquidity logic
if (activeTick < params.tickLower) {
    amount0Consumed = SqrtPriceMath.getAmount0Delta(
        sqrtRatioAX96,
        sqrtRatioBX96,
        liquidity,
        true
    );
} else if (activeTick < params.tickUpper) {
    amount0Consumed = SqrtPriceMath.getAmount0Delta(
        sqrtPriceX96,
        sqrtRatioBX96,
        liquidity,
        true
    );
    amount1Consumed = SqrtPriceMath.getAmount1Delta(
        sqrtRatioAX96,
        sqrtPriceX96,
        liquidity,
        true
    );
} else {
    amount1Consumed = SqrtPriceMath.getAmount1Delta(
        sqrtRatioAX96,
        sqrtPriceX96,
        liquidity,
        true
    );
}

Plan memory planner = Planner.init();
planner.add(
    Actions.CL_MINT_POSITION,
    abi.encode(
        params.poolKey,
        params.tickLower,
        params.tickUpper,
        uint256(liquidity),
        params.amount0In,
        params.amount1In,
        params.recipient,
        params.hookData
    )
);
bytes memory lockData = planner.finalizeModifyLiquidityWithSettlePair(
    params.poolKey

```

```
);

clPositionManager.modifyLiquidities{
  value: nativePair ? amount0Consumed : 0
}(lockData, deadline);
}
```

Remediation:

Calculate consumed amounts from balance changes around the Position Manager call and sweep any unused native currency back within the same plan before refunding it. Also calculate legacy withdrawal proceeds from balance changes instead of trusting values returned by caller-supplied contracts.

SQUAD1-2 | ManagerFeeHook.afterBurn returns the wrong selector

Fixed 

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

ManagerFeeHook.sol#L229

Description:

`afterBurn()` incorrectly returns the `afterMint` selector:

```
function afterBurn(...) external pure override returns (bytes4, BalanceDelta) {  
    ...  
    return (IBinHooks.afterMint.selector, BalanceDeltaLibrary.ZERO_DELTA);  
}
```

It should return `IBinHooks.afterBurn.selector`.

Currently, this does not cause an issue because `afterBurn` is not registered in `getHooksRegistrationBitmap()`, so the hook is never called.

However, if `afterBurn` is enabled in the future, `Hooks.callHook` will reject the callback, causing liquidity removals to revert.

Remediation:

Return `IBinHooks.afterBurn.selector`.

hexens x  **SQUAD
SWAP**

